

Where We Are Today

APPLICATION A • ArGO

1,800 tests in the regression suite

Each test runs for 6 user roles and on 2 databases.
That comes to more than 21,000 test runs in a full cycle.

600 tests are automated today. 600 more are planned.
The remaining 600 need physical hardware, so they stay manual.

APPLICATION B • AC-RT

~1,000 tests in the regression suite

The old automation setup here was outdated, so we rebuilt it from scratch on a newer technology.

20% of tests are already automated on the new setup, and we have also enabled the development team to add their own automated tests to it.

All of this is handled by a team of 3 QA engineers — along with manual testing, defect validation, and release support for every release.

CURRENT SITUATION

Why Tests Failed — and How We're Fixing Them

The recent release brought in important security fixes, and with them some screens and behavior changed. Our tests were written for the earlier screens, so some stopped passing. We have reviewed every failure — the causes are clear, and the fix is already underway.

What changed

Screen elements moved

Some buttons and fields were moved or renamed, so scripts look in the old place.

Upload / download timing

Files now take a different time; the old fixed waits no longer line up.

Pages refresh differently

Some pages reload in a new way, and scripts lose track of the page.

How we fix it

- 1 Fix locally.** Each engineer fixes and checks each feature on their own machine, with a report for each.
- 2 Merge & test together.** Fixes go to a shared branch; we run features alone and together to confirm they still work in combination.
- 3 Final integration.** Once stable, everything merges into the branch our scheduled runs use.
- 4 Verify on Jenkins.** We confirm a clean run on the shared server — so tests run automatically on every build, with no one starting them by hand.

Tracked throughout: each feature has its own fix story with a live status, backed by a shared sheet logging every failure, its cause, and where it stands.

Making the Suite Stronger Than Before

Alongside the fixes, we have prepared four improvements that make the automation more stable and easier to maintain going forward. All four are already scoped by the team.



Upgrade to Selenium 4.4

Moving to the current version of our test tool — more stable, better support for today's browsers, and the latest security fixes. Our older version is now behind, which adds avoidable flakiness.



Support both login types

The application now uses Keycloak single sign-on. The suite will handle both the normal login and the new SSO login, switchable by configuration — so tests keep working as login evolves.



Remove fixed waits

Old scripts pause for a fixed time; we're replacing these with smarter waits that respond to the page. Runs become faster and far more reliable on the shared server.



One codebase for versions 1.2 & 1.3

Both versions are live and differ slightly on screen. The suite will support both from a single codebase — so nothing is built or maintained twice.

How We Track the Fix

The stabilization work isn't run informally — it is broken down and tracked so progress is visible at any time.



A story per feature

- Every feature that needs fixing has its own individual story.
- Each story carries a live status, so it's clear what is done, in progress, or pending.
- Work is divided cleanly across the team — no overlap, nothing missed.



A shared analysis sheet

- Lists every feature that failed after the release.
- Records why it failed and the analysis behind it.
- A status column shows where each item stands, updated as work progresses.

The point: the fix is organized and measurable — anyone can see, at any moment, which features are recovered and which are still in progress.

One Risk We Want on the Table Now



A new design system is planned for the application.

When it arrives, screens will change again — and automated tests that work through the screens will need a round of rework, just like now. We are raising this today so that when the design system rollout is scheduled, a fix-up window for automation is scheduled along with it.

Suggested next step: since this will need rework on our side, we suggest a short discussion between the team driving the revamp and our QA trio once the design-system plans take shape — so the impact is understood early and the rework is planned together, not discovered later.

Catching Defects Earlier

Last release, most defects were found late, in UAT — the most expensive time to find them. Our aim is to move testing earlier, so problems surface while the work is still fresh. Two directions we are looking at:

1. Run automated checks before a build reaches QA

- When a build is ready, the existing automated tests run on the shared server before it is handed to QA — automatically, on every build.
- Obvious breakages are caught up front, so QA starts on a build that already works at a basic level.
- Low cost — it reuses tests we already have.

2. Add checks at the API level

- Screen-based tests can only catch what the screens show. Many deeper problems — in the application's logic and in how data moves between systems — don't show up through the screens.
- API checks test that logic directly, underneath the screens — reaching the kind of defect screen tests tend to miss.
- They are quick, run on every build, and a redesign does not break them.

Why both matter together: the first idea catches obvious breaks early. But screen-based testing alone — run by anyone — leaves the deeper defects for last. Adding API checks is how we start finding those earlier too, which is exactly the gap that hurt us last release.

Still a direction we are evaluating — not a finished plan. It can be taken up by QA, by the development team, or both together, and we will come back with a concrete proposal.